

面向对象程序设计

第03章 类与对象

赵毅力 ylzhao@swfu.edu.cn

大数据与智能工程学院

西南林业大学

Java语言中的引用类型

类型名称	类型含义
[]	数组
class	类
interface	接口
enum	枚举

面向对象程序设计原则

一切事物皆为对象。

任何系统（系统本身也是对象）均由对象构成。

系统的发展和进化均由系统内外部对象之间的相互作用完成。

面向对象程序设计

面向过程程序 = 数据结构 + 算法

面向对象程序 = 对象 + 类 + 继承 + 多态 + 消息传递

面向对象程序设计

对象(Object): 是类的实例, 由类构造产生。

消息(Message): 是对象之间的通信机制。不同对象之间通过消息传递的方式进行通信。

只有对象的系统是**静态**的, 加上对象之间的消息传递才使得系统成为**动态**的。

面向对象程序设计三个要素

封装

继承

多态

面向对象程序设计实例

- 实现一个扑克牌比较大小的游戏
 - 先比较两张扑克牌的花色：
 - 黑桃(Spade)>红桃(Heart)>方块(Diamond)>梅花(Club)
 - 如果两张扑克牌花色相同，再比较两张扑克牌的点数
 - $A > K > Q > J > 10 > 9 > 8 > 7 > 6 > 5 > 4 > 3 > 2$

面向对象分析和设计

针对具体问题，使用面向对象的方法对问题进行需求分析，称为“**面向对象分析**”。

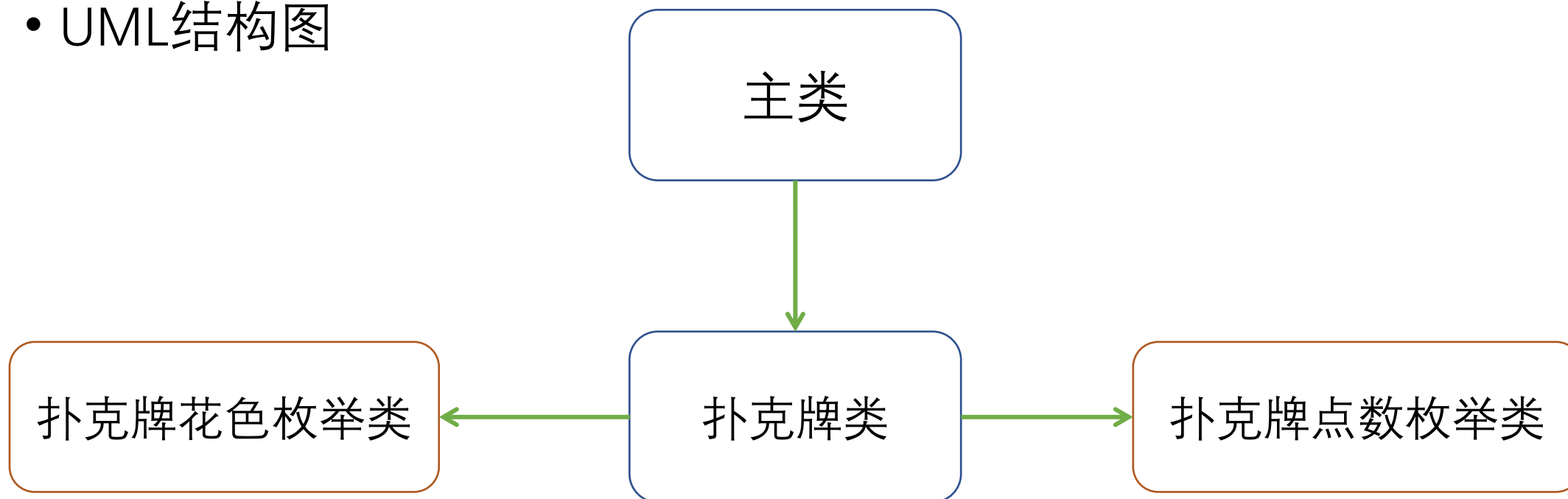
根据面向对象分析的结果，使用面向对象的方法对系统进行设计，称为“**面向对象设计**”。

根据面向对象设计的结果，使用面向对象程序设计语言进行实现称为“**面向对象实现**”。

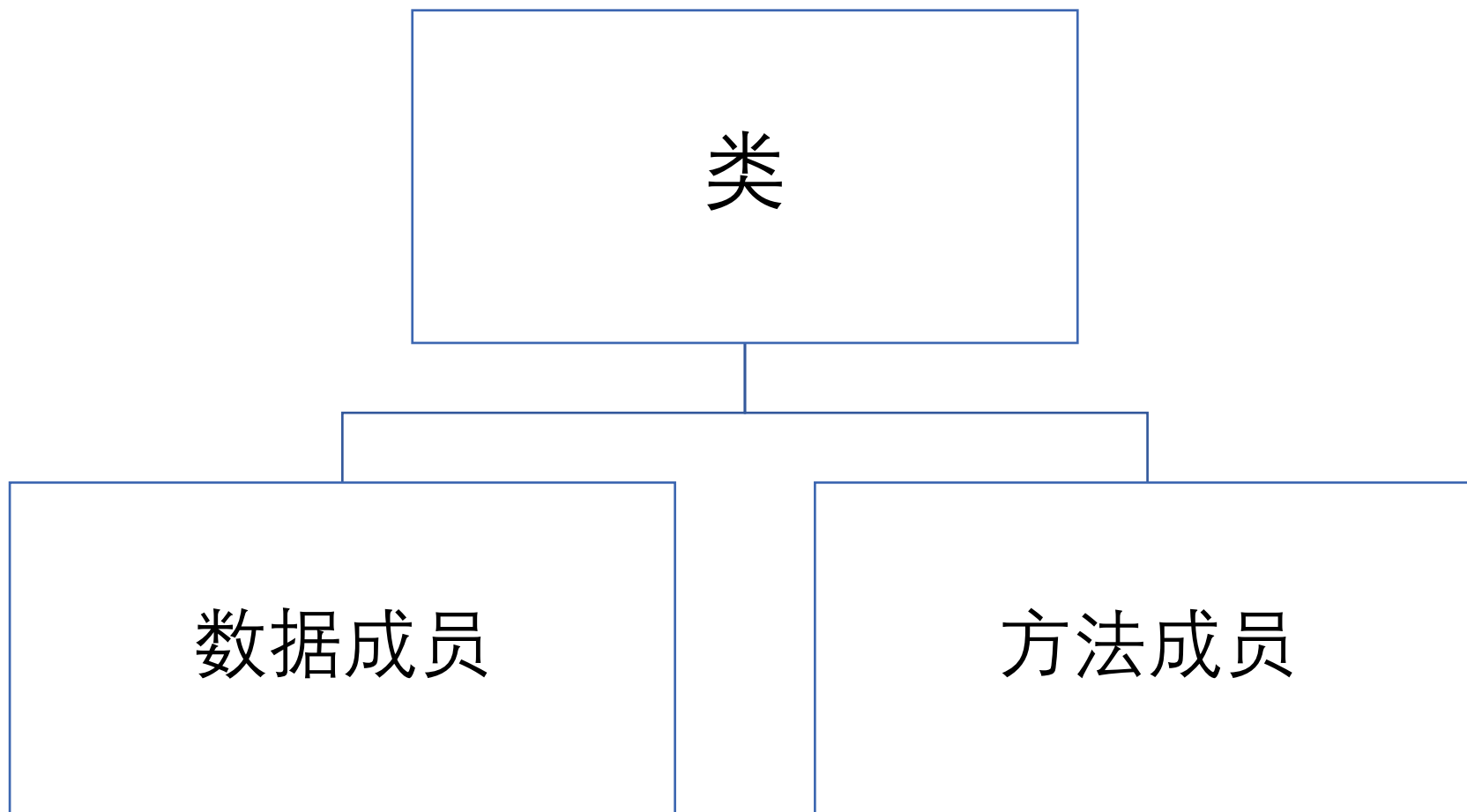
面向对象 = 面向对象分析 + 面向对象设计 + 面向对象实现

面向对象分析和设计

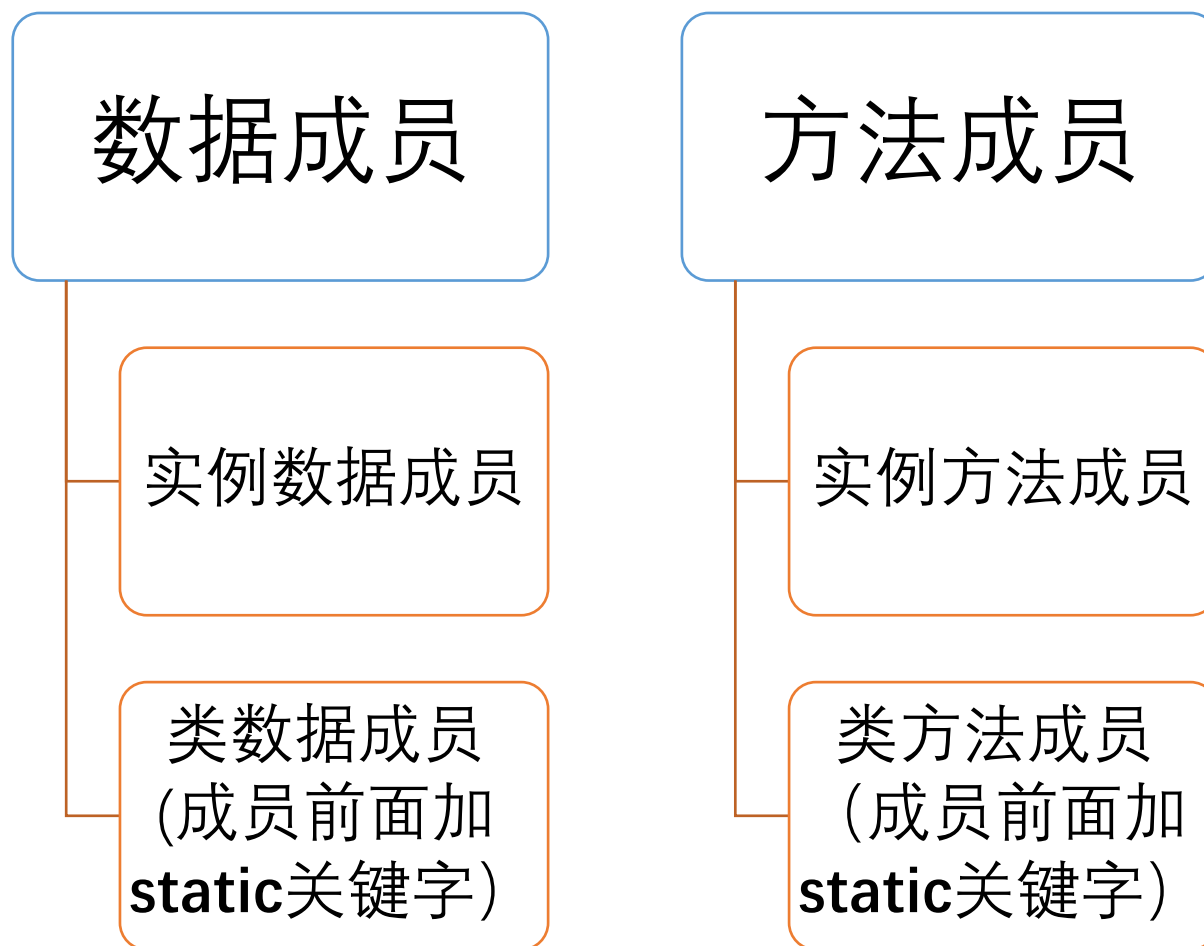
- UML结构图



类



类型成员

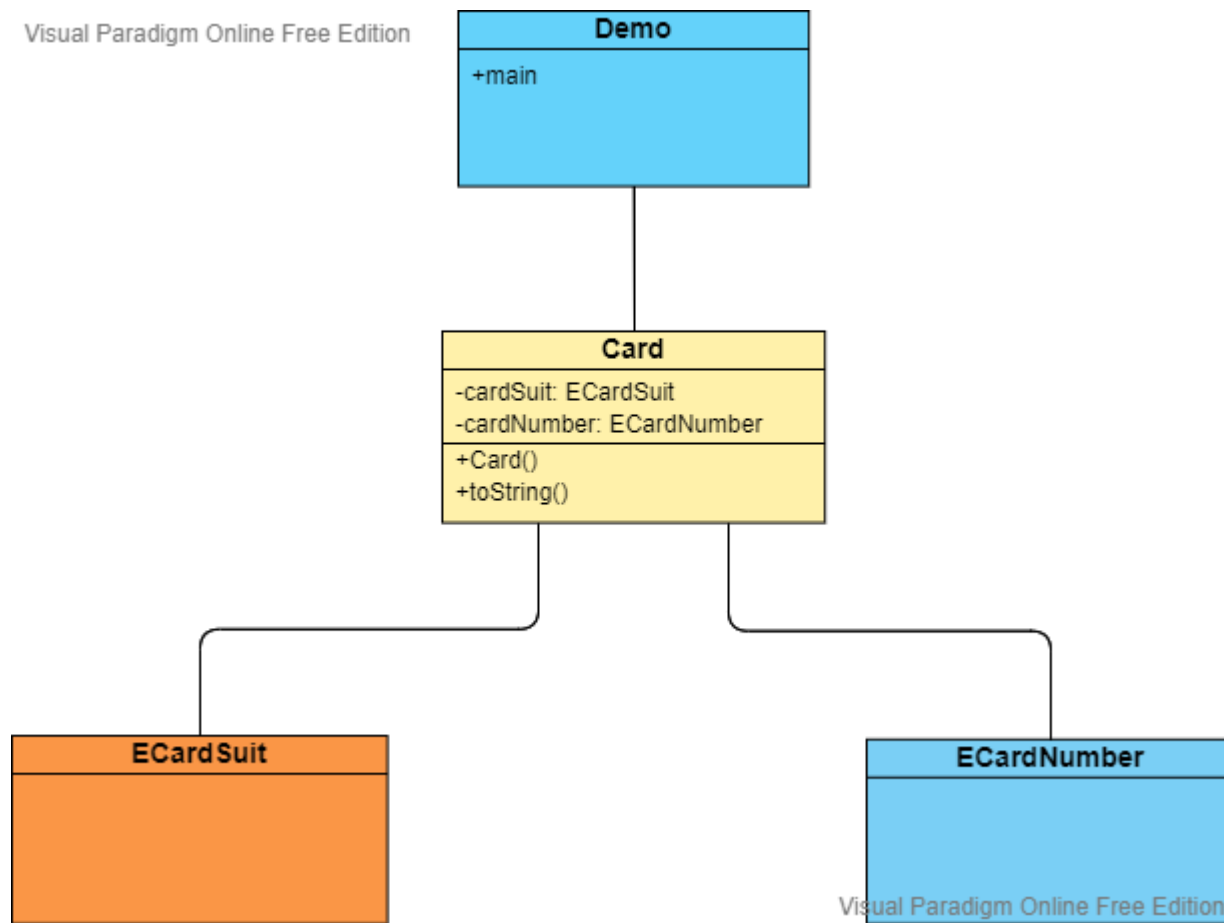


类成员的访问修饰符

访问修饰符	含义
private	私有访问
无	包访问
protected	保护访问
public	公有访问

系统UML图

Visual Paradigm Online Free Edition



类的定义

- Java语言使用关键字**class**定义一个类（每一个类都应该位于某个包中）：

```
package org.example;  
// 定义扑克牌类  
public class Card {  
    // 数据成员  
    // 方法成员  
}
```

- **Card**类保存的文件名称为**Card.java**
- 每一个.java文件里面**只能有一个类**声明为**public**

类的定义示例

- 定义一个扑克牌类
 - 数据成员
 - 花色（枚举类型变量）
 - 点数（枚举类型变量）
 - 方法成员
 - **构造方法**
 - 获取花色
 - 设置花色
 - 获取点数
 - 设置点数
 - ...

通过类创建对象实例

- Java语言使用new关键字创建类的对象实例

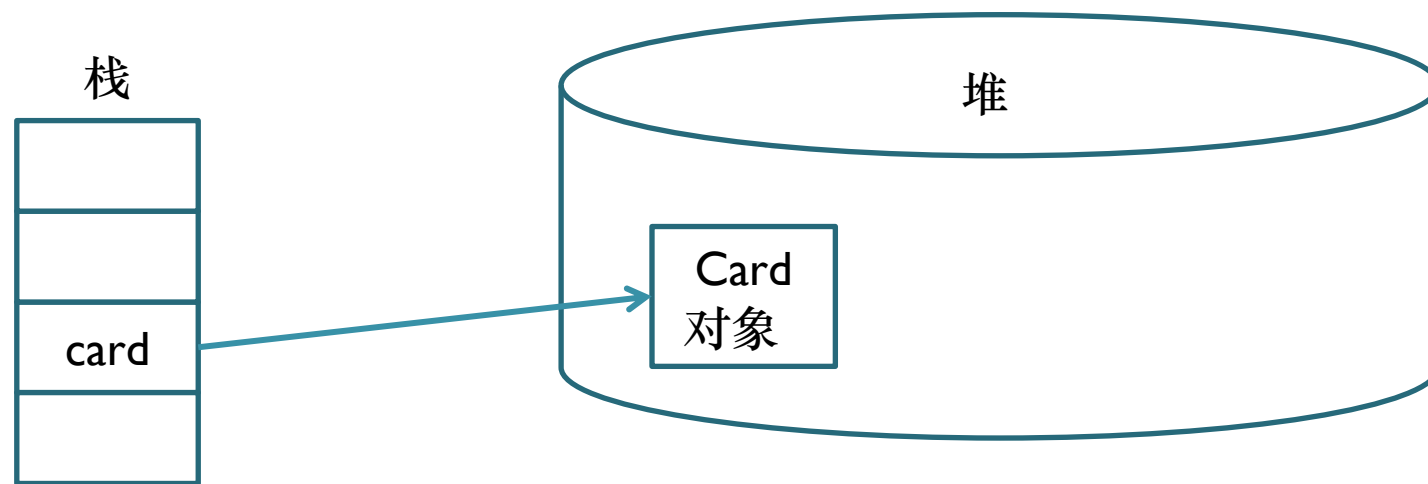
```
public class Card {  
    private ECardSuit cardSuit = ECardSuit.Club;  
    private ECardNumber cardNumber = ECardNumber.Two;  
  
    @Override  
    public String toString() {  
        return "Card{" +  
            "cardSuit=" + cardSuit +  
            ", cardNumber=" + cardNumber +  
            '}';  
    }  
}
```

```
package org.example;  
  
public class Main {  
    public static void main(String[] args) {  
        Card card = new Card();  
        System.out.println(card);  
    }  
}
```



通过类创建对象实例

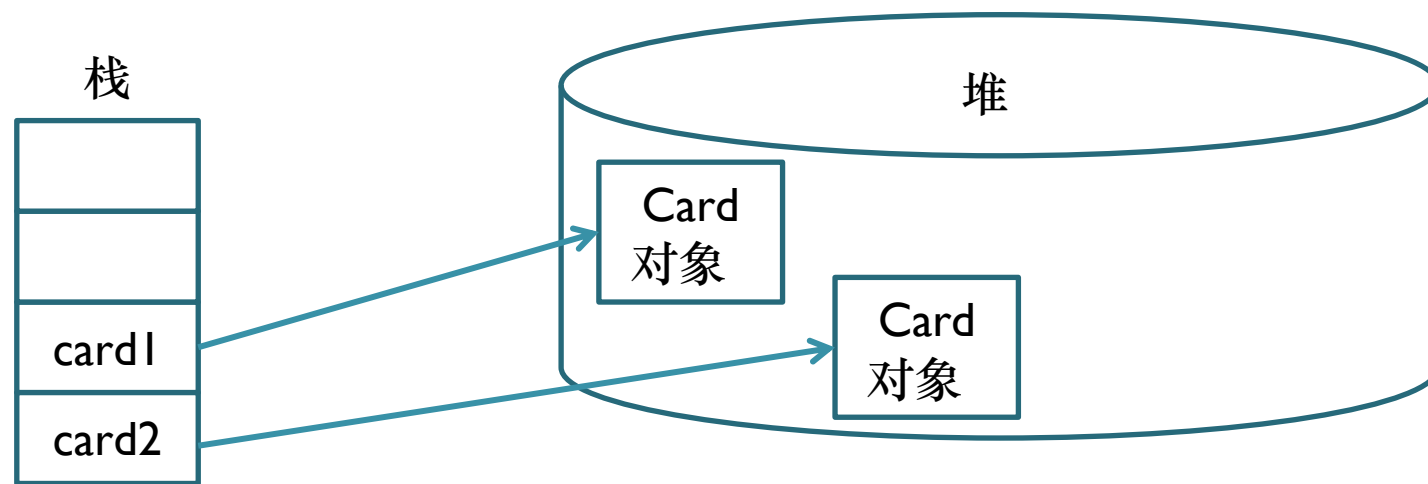
- `Card card = new Card();`



Card对象的内存空间分配在堆中，由虚拟机负责管理和回收。

通过类创建对象实例

- `Card card1 = new Card();`
- `Card card2 = new Card();`



Card对象的内存空间分配在堆中，由虚拟机负责管理和回收。

构造方法

- **构造方法**是一种特殊的成员方法，它的名字必须与它所在的**类的名字完全相同**，而且**没有返回类型**。
- 一个类可以包含**多个**构造方法，但必须保证参数各不相同。
- 任何时候只要创建该类的对象就会**自动调用该类的构造方法**。
- 构造方法其目的是用来**初始化**对象的各个成员变量，以便使创建的对象处于一个**合理的初始状态**。

缺省构造方法

- 如果没有为一个类定义构造方法，编译器会为**这个类合成一个构造方法**，称为**缺省构造方法**（default constructor）。

```
public class Card {  
    // 无构造函数的类  
}
```



```
public class Card {  
    public Card() {}  
}
```

- 系统合成的构造方法方法体为空，什么事情也不做。
- 如果用户为一个类定义了构造方法，编译器就不会再合成缺省构造方法了。(演示)

对象的创建和使用

```
Card card1 = new Card(); // 调用缺省构造方法
```

```
Card card2 = new Card(5, 3); // 调用带参数的构造方法
```

使用 **new** 表达式创建某个类的对象时，会使用构造方法创建和初始化所有实例成员变量，**该对象的内存分配在堆上，由虚拟机自动管理。**

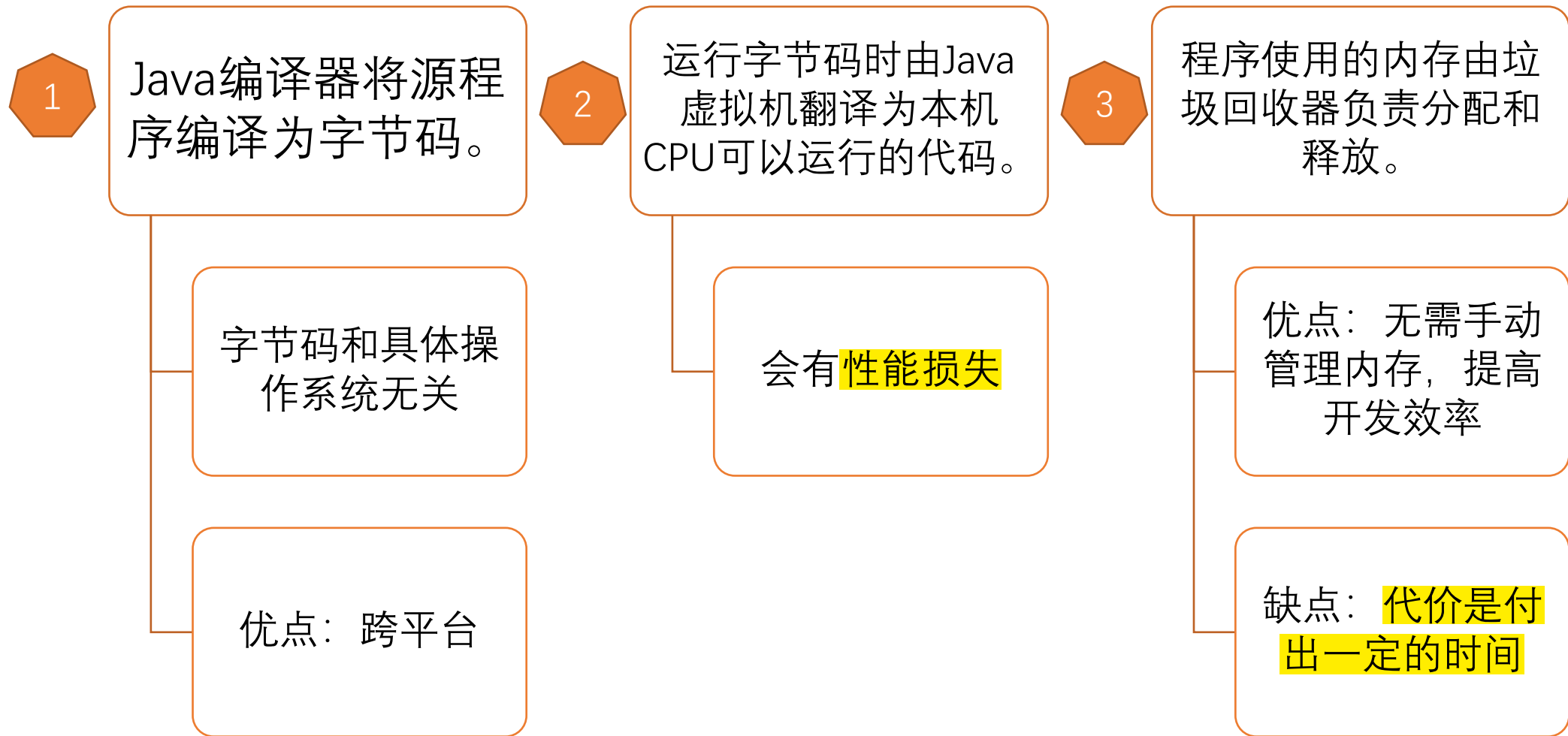
如果某个类没有定义构造方法，则系统会**自动生成一个默认构造方法**，并使用默认值来初始化对象的所有成员变量。

数据成员默认值

- 类的每个成员都有自己的默认值。

类型名称	默认值
byte	0
short	0
int	0
long	0L
char	'\0'
boolean	false
float	0.0F
double	0.0D
引用类型	null

托管代码和垃圾回收



类的使用(1)

- 如果需要使用的类和当前类在**同一个包**中，可以省略包的名称。

```
package org.example;
```

```
public class Util {  
    public compare() {  
        ...  
        ...  
    }  
}
```

```
package org.example;
```

```
public class Card {  
    public static void main(String[] args) {  
        Util util = new Util();  
        util.compare();  
    }  
}
```

类的使用(2)

- 如果需要使用的类和当前类不在同一个包中，就需要使用关键字 **import** 导入该类。

```
package org.demo;

public class Util {
    public int compare() {
        ...
        ...
        ...
        ...
    }
}
```

```
package org.example;

import org.demo.Util;

public class Card {
    public static void main(String[] args) {
        Util util = new Util();
        util.compare();
    }
}
```

参数传递

传递值类型参数

传递引用类型参数

传递值类型参数

- 演示

传递引用类型参数

- 演示

基本数据类型变量的比较

- 使用“==”运算符对基本数据类型进行比较

```
int x1 = 3;  
int x2 = 4;  
if (x1 == x2) {  
    // do something  
} else {  
    // do something  
}
```

- 如何判断两个浮点数相等？

引用类型变量的比较

- 引用类型变量存储的是**对象的地址**。

```
Card d1 = new Card();  
Card d2 = new Card();  
Card d3 = d1;
```

- d1 == d2 的值? d2 == d3 的值? d1 == d3 的值?
- 结论：使用“==”比较两个引用变量，是判别两个引用变量**是否指向相同的对象**。

字符串对象在意义上相等的比较

- 字符串对象的比较

```
String name1 = "fido";  
String name2 = "fido";
```

- name1 == name2 的值?

```
String name1 = new String("fido");  
String name2 = new String("fido");
```

- name1 == name2 的值?
- if (name1.equals(name2)) {} else {}
- 结论: 使用**equals**方法来比较字符串对象在意义上的相等。

随机生成一张扑克牌

- 随机生成一张扑克牌
 - 为**Card**类定义一个新的成员方法**randomize**
 - 花色**suit**成员：在**randomize**方法中从**ECardSuit**枚举类中随机选择一个成员
 - 点数**number**成员：在**randomize**方法中从**ECardNumber**枚举类中选择随机一个成员
- 枚举类型有一个**values**方法，以**一维数组**的形式返回该枚举类的所有成员。

如何对两张扑克牌对象进行比较？

- 扑克牌比较大小
 - 先比较两张扑克牌的**花色**:
 - 黑桃(Spade)>红桃(Heart)>方块(Diamond)>梅花(Club)
 - 如果两张扑克牌花色相同，再比较两张扑克牌的**点数**
 - $A > K > Q > J > 10 > 9 > 8 > 7 > 6 > 5 > 4 > 3 > 2$
- 实现**Comparable<T>**泛型接口
 - 需要实现接口中的**compareTo**方法
 - 如果方法返回-1，表示当前扑克牌对象比参数对象小；
 - 如果方法返回0，表示两个对象相等；
 - 如果方法返回1，表示当前扑克牌对象比参数对象大。

项目实例1

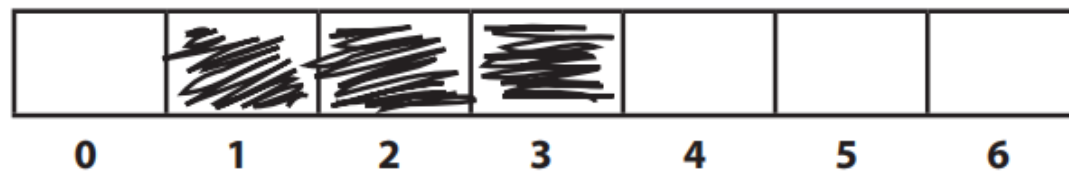
- 根据三角形的三条边a、b、c，计算对应的三角形面积。已知三角形的面积计算公式为：

$$area = \sqrt{s(s-a)(s-b)(s-c)}$$

其中 $s = (a + b + c)/2$ 。

项目实例2

- 请设计一个小游戏“Simple Dot Com”。游戏中每一个DotCom对象会占据7个单元格（0, 1, 2, 3, 4, 5, 6）中的**三个连续的单元格**（例如1, 2, 3），如下图所示。



- 系统在启动时**随机**为一个DotCom对象分配三个连续的单元格坐标，用户需要对DotCom对象占据的单元格坐标进行猜测。
 - (1) 如果没有猜中，输出字符串“**miss**”；
 - (2) 如果猜中，输出字符串“**hit**”；
 - (3) 如果DotCom所占据的三个单元格的坐标都被猜中，则输出“**dunk**”，并输出用户猜测的总次数。